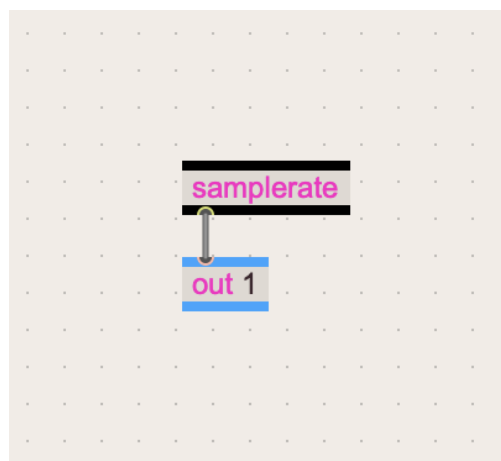


2. Modular (Arithmetic of) Time

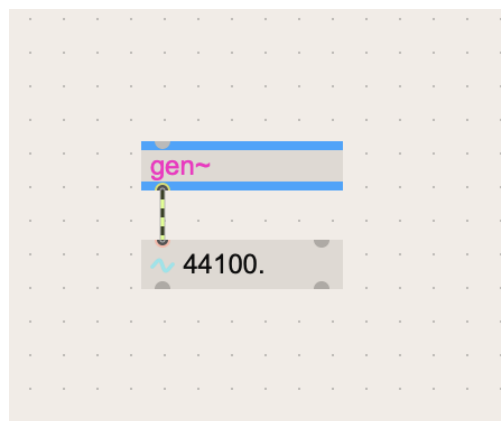
In `gen~`, the passing time of a signal is sliced into **sample frames** at a rate defined as the **samplerate**. The samplerate is the number of sample frames that pass per second.

Remember that `gen~` is calculating every passing sample frame. So think of the whole patch moving forward as a whole, one sample frame at a time.

There is a `samplerate` operator to see what the global sample rate is.



in `gen`



in `Max`

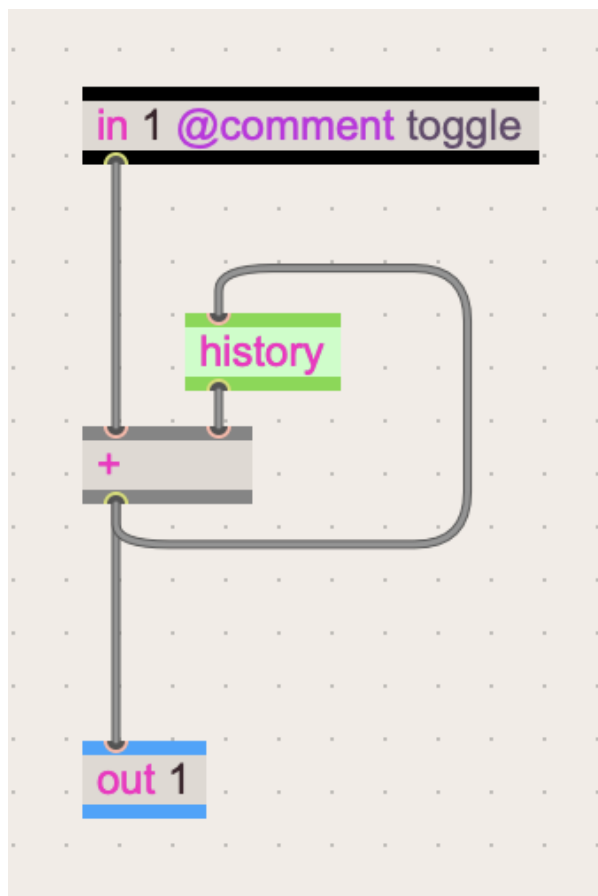
A simple counter

To create a simple counter, all we need are two objects:

- `[+]`
- `[history]`



Outside the gen patch shows a toggle in its first inlet, and a 'number' object for its output.



gen patch

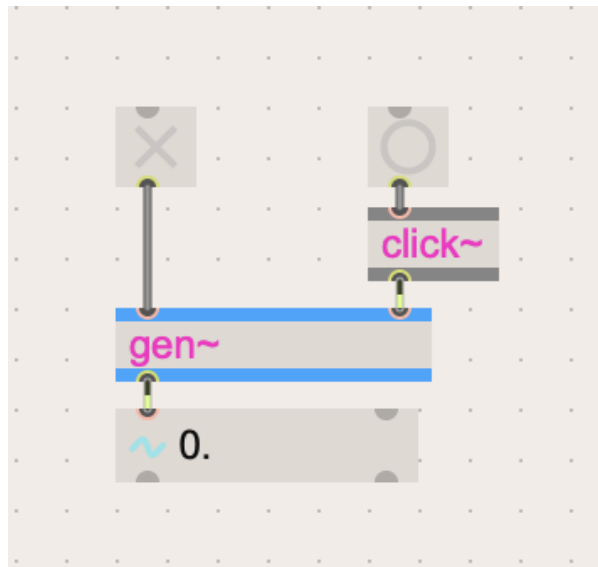
When the toggle is on, it sends a value of 1. When DSP is on, `[in 1]` will

send a 1, then it will add $1 + 0$, because the [history] object starts at 0, then sends it out to [out 1].

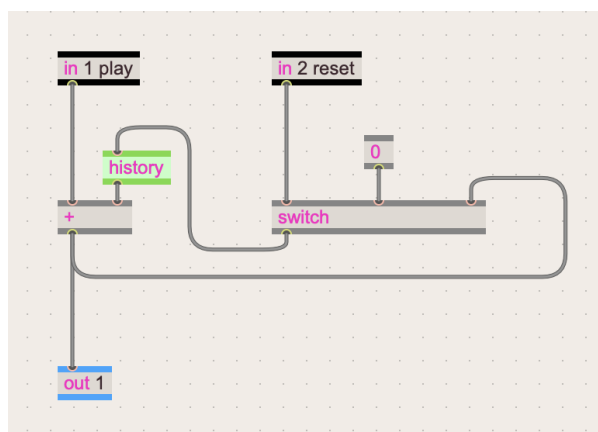
On the next sample frame, 1 comes from [in 1], and then it will do the addition of $1 + 1$, because 1 was stored in the [history] object last sample frame. It will send the 2 to [out 1], and will repeat. On the 48000th sample frame (if at a samplerate of 48k), it will accumulate to 48,000, after exactly 1 second.

Since we know that 48k is one second, we can also get the time elapsed by dividing it by [samplerate].

Adding a reset



The Max patch. Adding a 'click' (a one sample spike) will reset the count back to 0.



gen patch introduces a 'switch' operator.

The switch operator (?)

DESCRIPTION

Conditional Ternary Operator

Selects between the second and third inputs according to the boolean value of the first. If the first argument is true, the second argument will be output. Otherwise, the third argument will be output.

Same as the ? operator.

(inlet1: condition to test, inlet2: value if true, inlet3: value if false)

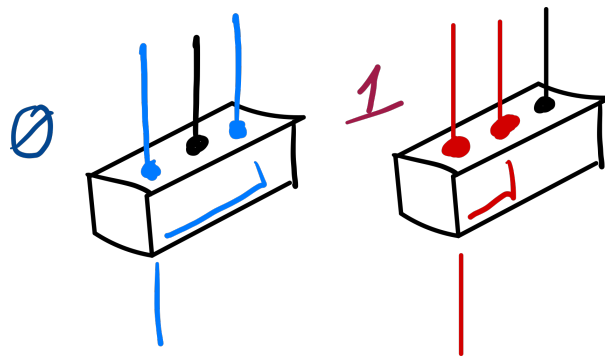


Illustration showing that switch allows 3rd inlet through when input is 0, and 2nd inlet when input is non-zero.

In code form:

```
switch = boolean_input ? inlet2 : inlet3;
```

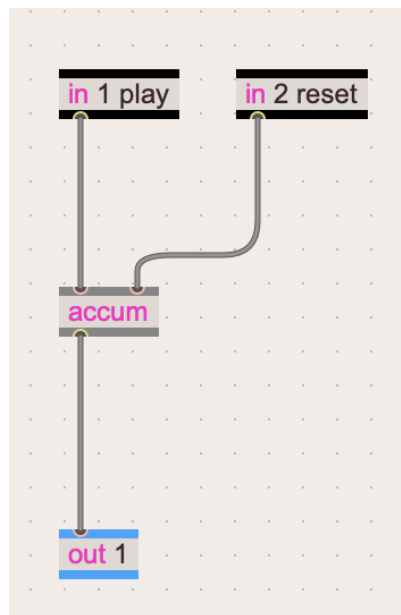
The accum operator

The patch we built is functions exactly like the accum operator.

DESCRIPTION**An Additive Accumulator**

The object adds to, and then outputs, an internal sum. This occurs at sample-rate, so the sum can grow very large, very fast. The value to be added is specified by either the first inlet or argument. The internal sum can be reset to the minimum by sending a nonzero value to the right-most inlet. The minimum value is 0 by default, but can be changed with the @min attribute. An optional maximum value can be specified with the @max attribute; values will wrap at the maximum.

(inlet1: amount to add, inlet2: nonzero reset)

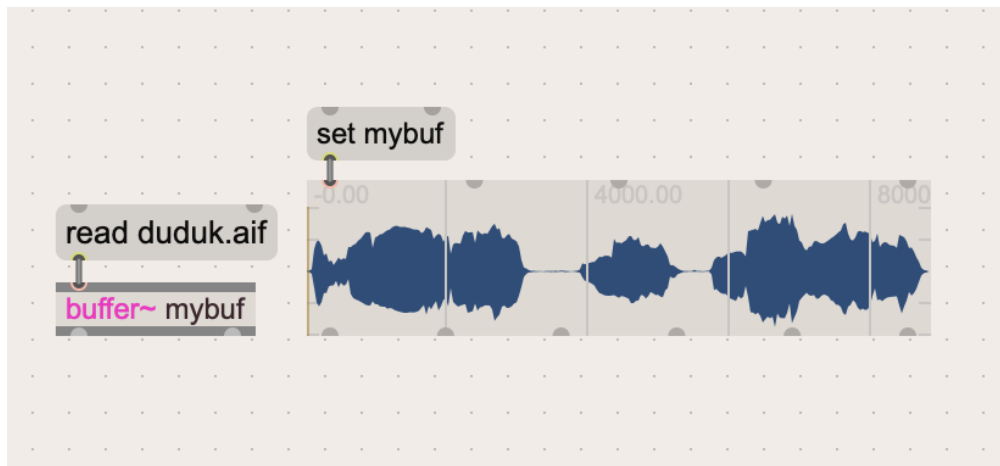


Same as the previous patch.

```
plusequals_1 = plusequals(in1, in2);  
out1 = plusequals_1;
```

Playing a Sound File

Let's load an audio file into a `buffer~`.



duduk.aif loaded into a buffer called 'mybuf'

Use the buffer operator to load it into `gen~`.

The buffer Operator

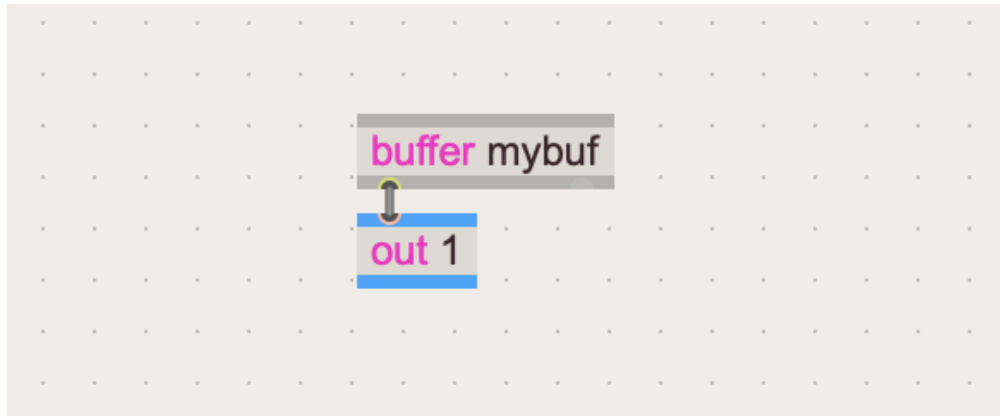
DESCRIPTION

A Reference to an External `buffer~` Object

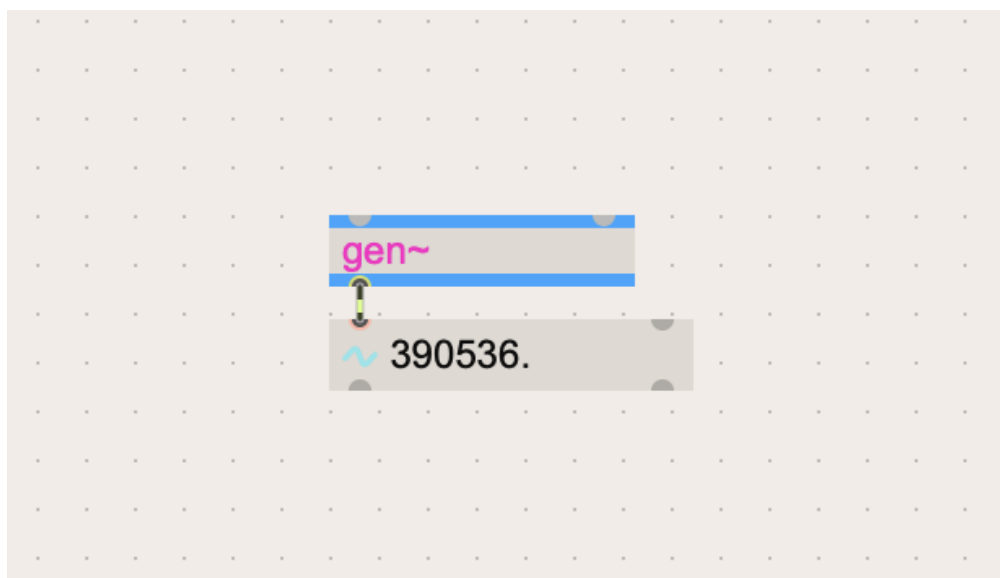
References an external named `buffer~` object. The first argument specifies a name by which to refer to this data in other objects in the `gen` patcher (such as `peek` and `poke`); the second optional argument specifies the name of the external `buffer~` object to reference (if omitted, the first argument name is used). The first outlet sends the length of the buffer in samples; the second outlet sends the number of channels.

arguments: (1. `buffer_name`)

(outlet1: length in samples, outlet2: number of channels)



gen patch showing the output of the first outlet of the buffer operator.



The max patch showing the number of samples in the buffer.

To read the data in the buffer in `gen~`, we can use the peek operator.

The peek operator

DESCRIPTION

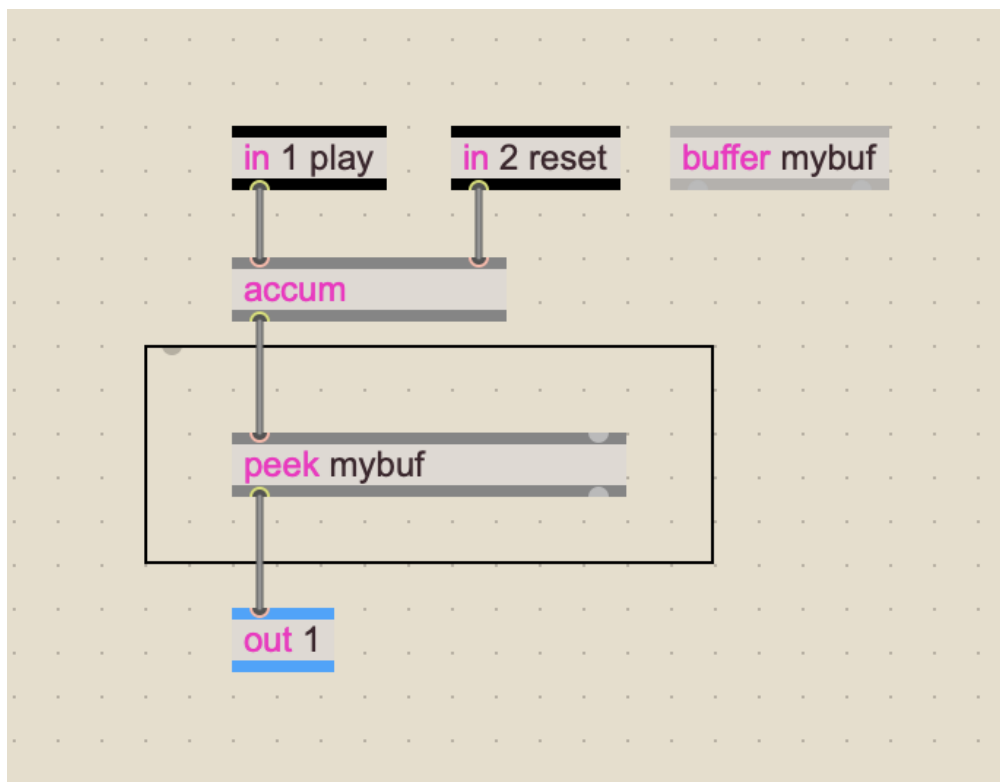
Read values from a data/buffer object

Read values from a data/buffer object. The first argument should be a name of a data or buffer object in the gen patcher. The second argument specifies the number of output channels. The first inlet specifies a sample index to read (no interpolation); indices out of range return zero. The last inlet specifies a channel offset (default 0).

arguments: (1. **name**, 2. **channels**)

outlets:

(1: **sample index to read**, 2: **channel_offset**)



gen patch with the peek operator.

This patch plays through the file once but it doesn't loop, because it just keeps accumulating until it is reset manually.

Looping a Buffer

We can have the buffer playback loop by using a 'wrap' operator.

The wrap operator

DESCRIPTION

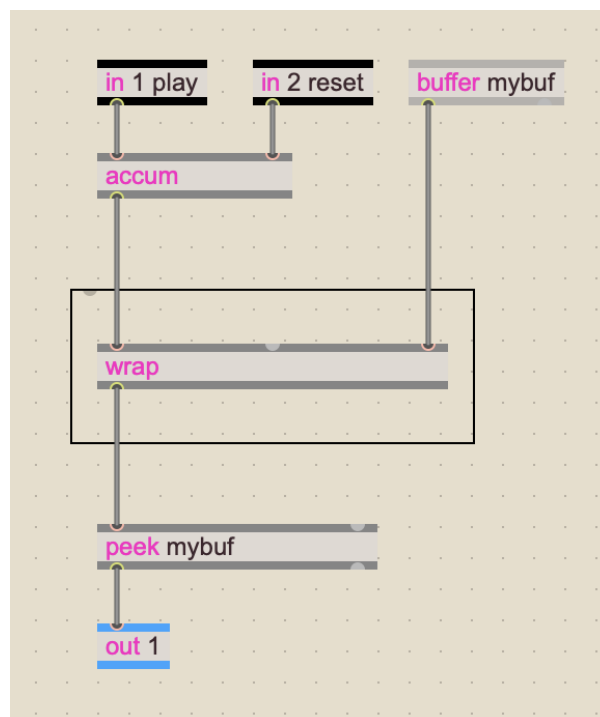
Wrap input to a range within a low and high output value

Low and high values can be specified by arguments or by inlets.
The default range is 0..1.

arguments: (1. input, 2. min, 3. max)

outlets:

(1: sample index to read, 2: channel_offset)



Adding the wrap with the max argument being provided by the buffer operator (giving the buffer sample size.)

Wavetable Synthesis

Let's understand the concept of wavetable synthesis by trying to design a Sin Wave Generator.

Here is the general formula:

$$s(t) = A \sin(2\pi ft + \phi)$$

or

$$A \sin(\omega t + \phi)$$

Where:

- **A: Amplitude**, the peak deviation of the function from zero.
- **f: frequency**, the number of oscillations (cycles) that occur each second of time.
- **ϕ : phase**, specifies (in radians) where in its cycle the oscillation is at $t = 0$.